Name:

UCInetID:

Data 275P: Fall 2026

Machine Learning with Generative Models

Homework 2

Due Date: **Friday, October 16th**

As with Homework 1, this homework includes a mix of code that should be submitted to the GradeScope autograder, and a templated PDF that should be filled out with mathematical derivations, plots, and numerical answers. Please see Canvas for more details, but as always please take care that:

- For the PDF submission, you must **fill in this PDF template** to provide your answers. All pages should remain in their original order when uploading your assignment to gradescope. *Do not rearrange, add, or remove any pages from the provided PDF template.*
- Enter your final answers in the dedicated answer spaces, and write your explanations and/or any math equations in the space provided below each question. *Always show the work needed to produce your answers, not just the final result.* Although typed responses are preferred, legible handwritten responses are fine.
- For multiple-choice questions, be sure to clearly select one of the answers, as well as providing an explanation or justification to support your answer.
- Check that all answers are readable, all scanned math or imported images are clear, and that content has not been cut off by the page margins. (We can only grade answers that we can see!)
- A Jupyter notebook version of the homework is also provided with each assignment.
- Some responses require inclusion of a plot or figure; after generating the plot in python, you can export it to a file or copy it to your clipboard, and then paste it into the homework PDF where indicated.
- If you would like to include code snippets in your responses, you can do the same thing – take an image of your code, and import it into the PDF in the space provided, or paste it into a textbox in the PDF.

If you have any questions/concerns about the homework problems or using Jupyter notebooks, ask us on EdD.

Summary of Assignment: 100 total points

- Problem 1: Linear Regression (30 + 15 points)
- Problem 2: Baysian Linear Regression (40 + 15 points)

Problem 1: Linear Regression (30 + 15 points)

In the next two problems, we study different approaches to linear regression using a one-dimensional dataset collected from a simulated motorcycle accident. The input variable, x, is the time in milliseconds since impact. The output variable, y, is the recorded head acceleration. We divide the full dataset into 40 training examples (`Xtrain`, `Ytrain`) and 53 test examples (`Xtest`, `Ytest`). To reduce numerical issues, all training features should be scaled to lie in the interval $[-1, +1]$ before fitting. Note that the same scaling must then be applied to all test data. We have provided example code to get you started.

We compare linear regression models with various families of non-linear basis functions. For a model of order L, we define $M = L + 1$ basis functions: the constant function $\phi_0(x) = 1$, and L functions $\phi_j(x), j = 1, \ldots, L$, that vary with x. In your solutions, you do not need to report the numerical values $\hat{w}$ of your estimated coefficients; we will assess correctness using the plots you create based on these estimates and your submitted code.

In the standard linear regression model, the observations $y^{(n)}$ follow a Gaussian distribution centered around a linear function w of a fixed set of basis functions:

$$p(y^{(n)}|x^{(n)}w, \beta) = \mathcal{N}(y^{(n)}|w^T\phi(x^{(n)}), \beta^{-1}).$$

Here, β is the inverse variance or precision. As derived in the textbook and lecture, given N training observations the maximum likelihood weights $\hat{w}$ satisfy the normal equations

$$(\Phi^T\Phi)\hat{w} = \Phi^T y,$$

where y is a N-dimensional vector of observed target values $y^{(n)}$, and Φ is an $N \times M$ matrix of corresponding features, such that $\Phi_{ij} = \phi_j(x^{(i)})$.

NOTE: For all parts below, you should implement your own numpy code for computing regression parameter estimates, rather than using standard packages for linear models.

1. Use the provided function `basis_poly` to define polynomial basis functions $\phi_j(x) = x^j$. Determine the maximum likelihood weight estimate $\hat{w}$ for a model of order $L = 5$. (You do not need to show anything in this part.)

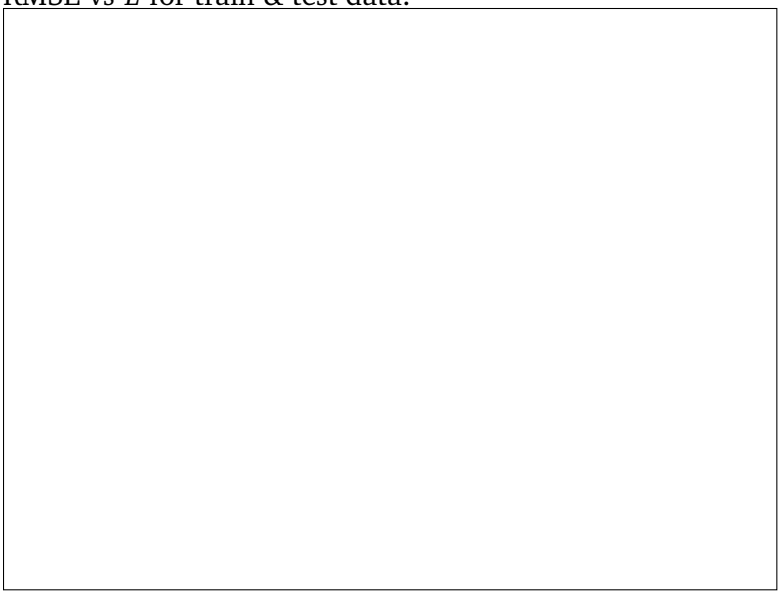
   Hint: To compute $x = A^{-1}b$ in Python, rather than explicitly calling the `np.linalg.inv` command, use the following command to improve numerical stability:
   `>> x = np.linalg.lstsq(A, b, rcond=-1)[0]`

2. Compute maximum likelihood (ML) estimates $\hat{w}$ of the regression parameters for models of order $L = 0, 1, 2, \ldots, 19$. Consider the root-mean squared error loss:

$$\mathcal{L}_{\text{rmse}}(x, y \mid \hat{w}) = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\left(y^{(n)} - \hat{w}^T\phi(x^{(n)})\right)^2} \tag{1}$$

   Evaluate and plot $\mathcal{L}_{\text{rmse}}(x, y \mid \hat{w})$ as a function of the model order, L, for the 40 training examples. Also do this (on the same plot axes) for the 53 test examples. Which model has the smallest training error, and which has the smallest test error?

RMSE vs L for train & test data:



Best Train RMSE, $L =$

Best Test RMSE, $L =$

On a single set of axes, using a dense grid of x values (see code), plot the test data and the mean predictions $\hat{w}^T \phi(x)$ for three models estimated via maximum likelihood: a model of order $L = 5$, the model with the best train error, and the model with the best test error.

Predictions & test data for your three models:



3. We now consider alternative, radial basis functions of the form

$$\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2\sigma^2}\right), \qquad j = 1, \ldots, L.$$

For any model order L, we space the basis function centers μ_j evenly between -1 and 1, and set the bandwidth $\sigma = (\mu_2 - \mu_1)$ to the distance between basis centers. See the provided function `basis_radial` in the jupyter notebook. Repeat parts (1)–(2) for radial basis function models of order $L = 0, 5, 10, 15, 20, 25$.

RMSE vs L for train & test data:

Best Train RMSE, $L =$

Best Test RMSE, $L =$

Predictions & test data for your three models:

4. In constructing the training and test sets, we excluded one point from the original dataset: $x = 57.6$, $y = 10.7$. What is the RMSE error in the prediction of this (single) point for the polynomial model that minimized the loss on the other test points? What is the error in the prediction of this point for the radial basis function model that minimized the loss on the other test points? Discuss differences between this datapoint and the other test data.

Loss, polynomial model:

Loss, radial basis model:

Discuss:

Problem 2: Bayesian Linear Regression (40 + 15 points)

In the previous question, you may have noticed that unregularized ML estimates can become unstable for large model orders. We now consider an alternative, Bayesian approach in which the regression coefficients are assigned a Gaussian prior

$$p(w) = \mathcal{N}(w \mid 0, \alpha^{-1} I_M)$$

where $\alpha > 0$ is an inverse-variance hyperparameter. As derived in the textbook, the posterior mean m_N and covariance S_N of the weight vector, under a Gaussian prior, equal

$$m_N = (\Phi^T \Phi + \frac{\alpha}{\beta} I_M)^{-1} \Phi^T y, \qquad S_N = (\beta \Phi^T \Phi + \alpha I_M)^{-1},$$

where I_M is an $M \times M$ identity matrix.

1. Under this model, the MAP, posterior mean, and posterior median estimators of w are all equivalent (and equal to m_N). For a polynomial basis of order $L = 50$ and $N = 40$ training examples, is the MAP estimate unique? Is the ML estimate unique? Explain your reasoning.

Is there a unique MAP estimate?
◯ Yes ◯ No

Is there a unique ML estimate?
◯ Yes ◯ No

Brief explanation:

2. Fix $\beta = 0.0025$, and consider 100 candidate values for the regularization parameter α, logarithmically spaced between 10^{-8} and $10^{0} = 1.0$:

```
»> alpha = np.logspace(-8,0,100)
```

Determine MAP estimates $\hat{w}$ for each of these priors, using a polynomial basis of order $L = 50$. Using the `pyplot.semilogx` command, plot the RMSE error metric (1) versus α for both the training and test datasets. Then plot the mean prediction $y(x) = \hat{w}^T \phi(x)$ along with the test data for the models that minimize the training and test error.

RMSE vs α for train & test data:

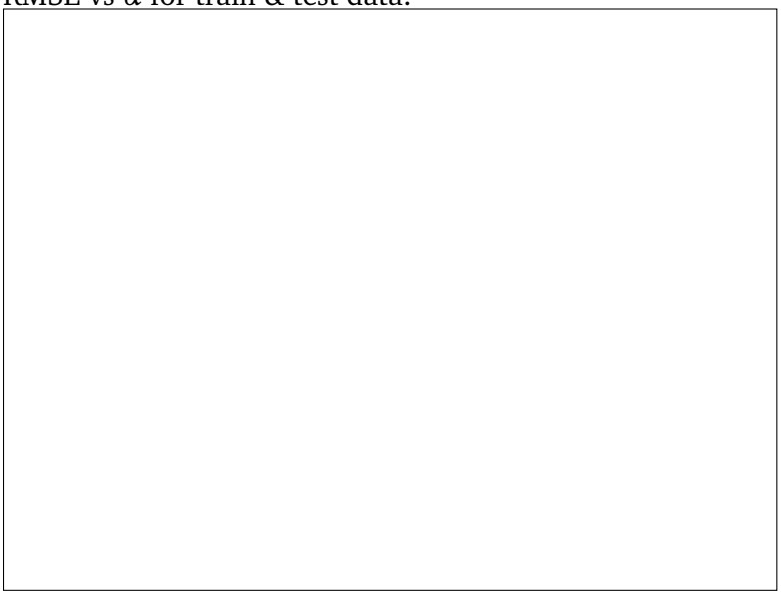
Best Train RMSE, $\alpha =$

Best Test RMSE, $\alpha =$

Predictions & test data for these two models:

3. Repeat part (b) for a radial basis function model of order $L = 50$.

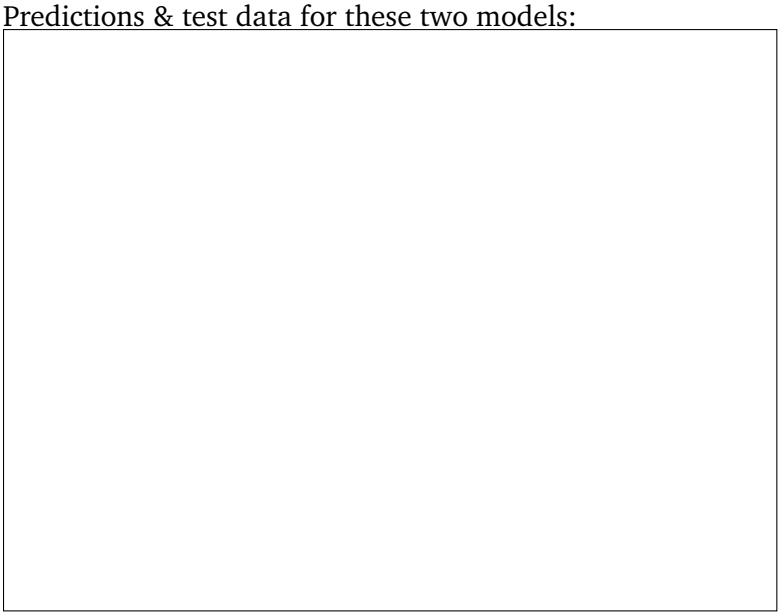
RMSE vs α for train & test data:



Best Train RMSE, $\alpha =$

Best Test RMSE, $\alpha =$

Predictions & test data for these two models:



4. Consider the pair of models from parts (b,c) that minimize the test error for their corresponding basis families. We can simulate from the posterior of the predictive function $\hat{y}(x)$ by first sampling weights $\tilde{w} \sim \mathcal{N}(w \mid m_N, s_N)$ from their posterior distribution given the training data, and then setting $\hat{y}(x) = \tilde{w}^T \phi(x)$. Draw 10 samples $\tilde{w}$ and plot their corresponding predictions $\hat{y}(x)$ for the polynomial and radial basis function features and posteriors, using the dense grid of x values defined in the example code.

   Hint: The numpy `np.random.multivariate_normal` command samples from a multivariate normal distribution.

Posterior sampled functions, polynomial regression:

Posterior sampled functions, radial basis regression:

5. Again consider the pair of models from part 2(d). What is the error in their corresponding predictions of the held-out test point from problem 1(d)? Are the relative magnitudes of these errors predictable from the posterior distributions plotted in part 2(d)?

Loss, polynomial model:

Loss, radial basis model:

Discuss: